

1 - Introduction to Delta Lake (Python)

[Import notebook](#)

databricks

Academy

01: Introduction to Delta Lake

This demonstration showcases Delta Lake's key features using the DataFrame API with Python:

- Setting up Delta tables
- Basic operations (`INSERT` , `UPDATE` , `DELETE`)
- `MERGE` operations
- Schema evolution
- Time travel capabilities
- Performance optimization

We'll be using data from the TPC-H dataset to demonstrate these concepts.

REQUIRED - SELECT CLASSIC COMPUTE

Before executing cells in this notebook, please select your classic compute cluster in the lab. Be aware that **Serverless** is enabled by default.

Follow these steps to select the classic compute cluster:

1. Navigate to the top-right of this notebook and click the drop-down menu to select your cluster. By default, the notebook will use **Serverless**.
2. If your cluster is available, select it and continue to the next cell. If the cluster is not shown:
 - In the drop-down, select **More**.

- In the **Attach to an existing compute resource** pop-up, select the first drop-down. You will see a unique cluster name in that drop-down. Please select that cluster.

NOTE: If your cluster has terminated, you might need to restart it in order to select it. To do this:

1. Right-click on **Compute** in the left navigation pane and select *Open in new tab*.
2. Find the triangle icon to the right of your compute cluster name and click it.
3. Wait a few minutes for the cluster to start.
4. Once the cluster is running, complete the steps above to select your cluster.

A. Classroom Setup

Run the following cell to configure your working environment for this course. It will also set your default catalog to **dbacademy** and the schema to your specific schema name shown below using the `USE` statements.

```
USE CATALOG dbacademy;  
USE SCHEMA dbacademy.<your unique schema name>;
```

NOTE: The `DA` object is only used in Databricks Academy courses and is not available outside of these courses. It will dynamically reference the information needed to run the course.

```
%run ../Includes/Classroom-Setup-Common
```

Note: you may need to restart the kernel using `%restart_python` or `dbutils.library.restartPython()` to use updated packages.

B. Creating Delta Tables

Delta tables can be created using standard DDL, e.g. `CREATE TABLE` statements (which can be used to create a new empty delta table), these can also be `CREATE TABLE AS SELECT` to create tables from the results of a query.

NOTE: Delta tables support the `CREATE OR REPLACE TABLE ... AS SELECT` feature which allows a table to be redefined with the results of the new `SELECT` statement.

Let's create our first Delta table using data from the TPC-H dataset. We'll start with the **orders** table.

```
from pyspark.sql.functions import col
order_df = spark.read.table("samples.tpch.orders")
```

▶  order_df: pyspark.sql.dataframe.DataFrame = [o_orderkey: long, o_custkey: long ... 7 more fields]

```
# Displaying Orders_df
display(order_df)
```

Table

```

spark.sql("DROP TABLE IF EXISTS delta_orders")

# Transform and save the orders data using DataFrame API
orders_df = (
    order_df.select(
        col("o_orderkey").alias("order_id"),
        col("o_custkey").alias("customer_id"),
        col("o_orderstatus").alias("status"),
        col("o_totalprice").alias("total_price"),
        col("o_orderdate").alias("order_date"),
        col("o_orderpriority").alias("priority"),
        col("o_clerk").alias("clerk"),
        col("o_shippriority").alias("ship_priority"),
        col("o_comment").alias("comment")
    )
    .filter(col("o_orderdate") >= "1996-01-01")
    .limit(10000)
)

# Write the DataFrame as a Delta table
orders_df.write.mode("overwrite").saveAsTable("delta_orders")

# Display the Delta table
display(spark.table("delta_orders"))

```

►  orders_df: pyspark.sql.dataframe.DataFrame = [order_id: long, customer_id: long ... 7 more fields]

Table

NOTE: Delta Lake is the default format for DataBricks runtimes, otherwise you would need to use:

```
orders_df.write.format("delta").mode("overwrite").saveAsTable("delta_orders")
```

```
%sql
-- The SQL equivalent of 👍
DROP TABLE IF EXISTS delta_orders_sql;

CREATE OR REPLACE TABLE delta_orders_sql
AS
SELECT
  o_orderkey AS order_id,
  o_custkey AS customer_id,
  o_orderstatus AS status,
  o_totalprice AS total_price,
  o_orderdate AS order_date,
  o_orderpriority AS priority,
  o_clerk AS clerk,
  o_shippriority AS ship_priority,
  o_comment AS comment
FROM samples.tpch.orders
WHERE o_orderdate >= '1996-01-01'
LIMIT 10000;

SELECT * FROM delta_orders_sql;
```

►  _sqldf: pyspark.sql.dataframe.DataFrame = [order_id: long, customer_id: long ... 7 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

```
%sql
-- Inspect table metadata
DESCRIBE EXTENDED delta_orders
```

▸  `_sqldf`: `pyspark.sql.dataframe.DataFrame` = [col_name: string, data_type: string ... 1 more field]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

```
# Inspecting using spark.sql
display(spark.sql("DESCRIBE EXTENDED delta_orders"))
```

Table

```
%sql
-- This is a delta lake specific command which returns detailed metadata about
a Delta table including statistics
DESCRIBE DETAIL delta_orders
```

▸  _sqldf: pyspark.sql.dataframe.DataFrame = [format: string, id: string ... 14 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

```
%sql
-- Use this command to see the history of changes to the delta table, we will
revisit this later..
DESCRIBE HISTORY delta_orders
```

►  `_sqldf`: pyspark.sql.dataframe.DataFrame = [version: long, timestamp: timestamp ... 13 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

```
# We can use the DeltaTable class to interact with the Delta table
programmatically
from delta.tables import DeltaTable

# Get the Delta table
delta_table = DeltaTable.forName(spark, "delta_orders")

display(delta_table.history())
```

Table

C. Basic Delta Operations with DataFrame API

Let's demonstrate basic DML operations with Delta tables using the DataFrame API.

1. Insert Operation

```
# Create a DataFrame with new records
from datetime import date
from decimal import Decimal

new_orders = spark.createDataFrame([
    (999997, 12345, 'P', Decimal('1234.56'), date(2023, 2, 15), "1-URGENT",
    "Clerk#000000123", 0, "New test order using Python"),
    (999996, 12346, 'O', Decimal('5678.90'), date(2023, 2, 16), "2-HIGH",
    "Clerk#000000456", 0, "Another test order using Python")
], schema=orders_df.schema)

# Append new records to the Delta table
new_orders.write.format("delta").mode("append").saveAsTable("delta_orders")

# Verify the new records
display(spark.table("delta_orders").filter(col("order_id").isin([999997,
999996])))
```

►  new_orders: pyspark.sql.dataframe.DataFrame = [order_id: long, customer_id: long ... 7 more fields]

Table

```
%sql
-- Insert two more records using SQL
INSERT INTO delta_orders
VALUES
(999999, 12345, 'P', DECIMAL('1234.56'), DATE('2023-02-15'), '1-URGENT',
'Clerk#000000123', 0, 'New test order using SQL'),
(999998, 12346, 'O', DECIMAL('5678.90'), DATE('2023-02-16'), '2-HIGH',
'Clerk#000000456', 0, 'Another test order using SQL');

-- Verify the insertion
SELECT * FROM delta_orders WHERE order_id IN (999999, 999998);
```

►  _sqldf: pyspark.sql.dataframe.DataFrame = [order_id: long, customer_id: long ... 7 more fields]

Table

 This result is stored as _sqldf and can be used in other Python and SQL cells.

```
# Check the history again now
display(delta_table.history())
```

Table

2. DELETE Operation

```
%sql
-- Whoops!
DELETE FROM delta_orders
```

►  _sqldf: pyspark.sql.dataframe.DataFrame = [num_affected_rows: long]

Table

 This result is stored as _sqldf and can be used in other Python and SQL cells.

```
# Check the history again now
display(delta_table.history())
```

Table

```
# Check the delta dataframe count
delta_table.toDF().count()
```

0

D. Time Travel Operations

Explore Delta Lake's time travel capabilities for data auditing and recovery. Let's use this feature to query previous versions of our data. **Time Travel can be performed using:**

- **Version numbers:** Integer values starting from 0 (e.g., `VERSION AS OF 2`)
- **Timestamps:** Formatted as ISO-8601 strings (e.g., `TIMESTAMP AS OF '2023-02-15T12:30:00.000Z'`)

```
# Read a specific version of the Delta table
previous_version = spark.read.option("versionAsOf", 2).table("delta_orders")
display(previous_version.filter(col("order_id").isin([999999, 999998, 999997, 999996])))
```

▶  previous_version: pyspark.sql.dataframe.DataFrame = [order_id: long, customer_id: long ... 7 more fields]

Table

```
%sql
-- The SQL equivalent of 👍
SELECT * FROM delta_orders VERSION AS OF 2
WHERE order_id IN (999999, 999998, 999997, 999996);
```

▶  _sqldf: pyspark.sql.dataframe.DataFrame = [order_id: long, customer_id: long ... 7 more fields]

Table

 This result is stored as _sqldf and can be used in other Python and SQL cells.

```
%sql
DESCRIBE HISTORY delta_orders
```

▶  _sqldf: pyspark.sql.dataframe.DataFrame = [version: long, timestamp: timestamp ... 13 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

Replace with an actual timestamp of version from your `DESCRIBE HISTORY` results.
Format example: '2023-02-15T12:30:00.000Z' or '2023-02-15 12:30:00'

```
# We can also traverse historical versions using a timestamp
timestamp_to_restore_to = '2025-07-02T16:42:39.000+00:00'
display(
    spark.read.option("timestampAsOf",
timestamp_to_restore_to).table("delta_orders")
)
```

Table

NOTE: Use the command `describe history <table_name>` to get version info.

You will find the transaction time to use above.

E. Restore Operations

Learn how to recover data using Delta Lake's restore capabilities.

```
# Restore to the original version using SQL
spark.sql("RESTORE TABLE delta_orders TO VERSION AS OF 1")

# Verify the restore worked
display(spark.table("delta_orders"))
```

Table

```
# Check the history again now
display(delta_table.history())
```

Table

F. Using MERGE Function

Delta Lake provides powerful **MERGE** capabilities for upsert operations (update + insert) - as well as conditional `DELETE`s. Before jumping to MERGE lets look at Upsert functionality

What is an Upsert?

- An "upsert" combines UPDATE and INSERT operations in a single atomic transaction
- If a matching record exists, it's updated; if not, a new record is inserted
- This is particularly useful for scenarios like:
 - Processing change data capture (CDC) feeds
 - Handling slowly changing dimensions (SCD)
 - Streaming data integration
 - Deduplication with latest values

Let's see how Upsert can be used in MERGE operation

```
%sql
-- First, create a temporary view with updated order data
CREATE OR REPLACE TEMPORARY VIEW updated_orders AS
SELECT 999996 AS order_id, 12346 AS customer_id, 'O' AS status,
       CAST(6000.00 AS DECIMAL(18,2)) AS total_price, -- Updated price
       CAST('2023-02-16' AS DATE) AS order_date,
       '1-URGENT' AS priority, -- Changed from 2-HIGH to 1-URGENT
       'Clerk#000000456' AS clerk,
       0 AS ship_priority,
       'Updated order with higher priority' AS comment -- Updated comment
UNION ALL
-- Add a new record that doesn't exist in the table yet
SELECT 999995 AS order_id, 12347 AS customer_id, 'P' AS status,
       CAST(2500.50 AS DECIMAL(18,2)) AS total_price,
       CAST('2023-02-17' AS DATE) AS order_date,
       '3-MEDIUM' AS priority,
       'Clerk#000000789' AS clerk,
       1 AS ship_priority,
       'New order via MERGE' AS comment;

-- Now perform the MERGE operation
MERGE INTO delta_orders
USING updated_orders AS updates
ON delta_orders.order_id = updates.order_id
WHEN MATCHED THEN
    UPDATE SET
        total_price = updates.total_price,
        priority = updates.priority,
        comment = updates.comment
WHEN NOT MATCHED THEN
    INSERT (order_id, customer_id, status, total_price, order_date, priority,
           clerk, ship_priority, comment)
    VALUES (updates.order_id, updates.customer_id, updates.status,
            updates.total_price,
                updates.order_date, updates.priority, updates.clerk,
            updates.ship_priority, updates.comment);

-- Verify the results
SELECT * FROM delta_orders WHERE order_id IN (999995, 999996);
```

►  _sqldf: pyspark.sql.dataframe.DataFrame = [order_id: long, customer_id: long ... 7 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

NOTE: The upsert is performed by the MERGE statement, which updates existing rows and inserts new ones based on whether the `order_id` exists in the target table (`delta_orders`)

```
%sql
-- See the MERGE operation in the table history
DESCRIBE HISTORY delta_orders
```

►  `_sqldf`: `pyspark.sql.dataframe.DataFrame` = [version: long, timestamp: timestamp ... 13 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

G. Optimizing File Sizes with OPTIMIZE

Delta Lake tables can accumulate many small files over time, especially after numerous `UPDATE`, `DELETE`, or `MERGE` operations. The `OPTIMIZE` command helps maintain performance by compacting these small files into larger ones, reducing the metadata that needs to be processed during queries.

Note that Databricks automatically applies optimized writes to Delta tables by default in newer runtimes, which helps reduce small file issues proactively.

```
%sql
-- Note the value of the NumFiles column
DESCRIBE DETAIL delta_orders
```

▸  `_sqldf`: pyspark.sql.dataframe.DataFrame = [format: string, id: string ... 14 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

```
%sql
-- Optimize the delta_orders table to improve query performance
OPTIMIZE delta_orders
ZORDER BY (order_date, customer_id);

-- Check optimization metrics
DESCRIBE HISTORY delta_orders;
```

▸  _sqldf: pyspark.sql.dataframe.DataFrame = [version: long, timestamp: timestamp ... 13 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

```
%sql
-- Now look at the NumFiles column again
DESCRIBE DETAIL delta_orders
```

▸  _sqldf: pyspark.sql.dataframe.DataFrame = [format: string, id: string ... 14 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

H. Removing Older Versions using VACUUM

Delta Lake maintains a history of changes, allowing time travel queries. However, this can consume storage space over time. The `VACUUM` command permanently removes files that are no longer referenced by the latest versions of the table.

```
%sql
-- First, check the retention period (default is 7 days)
SET spark.databricks.delta.retentionDurationCheck.enabled = false;

-- Remove files no longer referenced by the Delta table and are older than the
retention period
-- See what would be deleted without actually removing files (dry run)
VACUUM delta_orders RETAIN 24 HOURS DRY RUN;
```

►  `_sqldf: pyspark.sql.dataframe.DataFrame = [path: string]`

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

```
%sql
-- Now do it for real...
-- WARNING: This will permanently remove access to older versions
VACUUM delta_orders RETAIN 24 HOURS;
```

►  _sqldf: pyspark.sql.dataframe.DataFrame = [path: string]

Table

 This result is stored as _sqldf and can be used in other Python and SQL cells.

```
%sql
-- Note that VACUUM operations appear in the log as well
DESCRIBE HISTORY delta_orders
```

►  _sqldf: pyspark.sql.dataframe.DataFrame = [version: long, timestamp: timestamp ... 13 more fields]

Table

 This result is stored as `_sqldf` and can be used in other Python and SQL cells.

Databricks provides automatic VACUUM behavior:

- Default retention period is 7 days (168 hours)
- Delta tables are automatically vacuumed according to this retention period
- Safety checks prevent accidental removal of versions less than 7 days old

NOTE: Vacuumed version may still be available after a `VACUUM` operation, this is due to **delta caching**, however don't rely on this, the version files could get evicted from delta cache at any stage (and will not longer be available after the cluster restarts)

Key Takeaways

1. Versioning and Time Travel

- All changes create new versions
- Access historical versions easily
- Query data as of specific timestamps

2. Data Recovery

- Restore to previous versions
- Audit data changes
- Track modifications

3. Optimization

- Regular table optimization
- Z-ORDER indexing
- File compaction

4. Maintenance

- Implement regular cleanup
- Update statistics

- Monitor table health

© 2025 Databricks, Inc. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation (<https://www.apache.org/>).

Privacy Policy (<https://databricks.com/privacy-policy>) | Terms of Use (<https://databricks.com/terms-of-use>) | Support (<https://help.databricks.com/>)